Node-Local Deploy Queues and the `f1r3node-async-rust` Client

A design note for `f1r3node-rust`

L. G. Meredith
F1R3FLY.io

7 November 2026 (surveyed at `dev` `a79a6ce`, `feature/rev-stacks-ote` `4c9853b`)

Abstract

A shard should absorb a burst of deployments from many clients without stalling. This note designs two components that make that so: a pairing of each `f1r3node` with its own RabbitMQ queue, drained by a consumer task inside the node that hands each message to the existing deploy path; and an async Rust client that signs a deploy, publishes it to a node's queue, and follows its fate through status events. The queue is strictly node-local, so no shared service can take the network down. The deploy's block-window validity stays as the mechanism that bounds a backlog; it is already a shard-wide genesis parameter on both branches surveyed, so no new consensus parameter is needed. Everything branch-specific sits behind one seam, `BlockAPI::deploy`, which the design does not cross, so one implementation serves current `dev` and the cost-accounting branch intended for the ASI TestNet.

Contents

1 Aim and scope **1**

2 Where this sits in the architecture **2**

3 Today's deploy ingress on `dev` **3**

4 What changes on the cost-accounting branch **4**

5 Component 1: the node + RabbitMQ pairing **6**

6 Deploy TTL as a shard-wide parameter **8**

7 Component 2: the `f1r3node-async-rust` client **10**

8 Branch compatibility **12**

9 Operations **12**

10 Development plan **14**

11 Open questions and decisions needed **15**

1 Aim and scope

Two components deliver burst tolerance:

1. **Node + queue pairing.** Every `f1r3node` in the shard runs beside its own RabbitMQ broker. A consumer task inside the node receives messages from that queue as they arrive—RabbitMQ pushes deliveries to a registered consumer, the node never polls—and hands each one to the node's existing deploy path, exactly as if it had come from the `f1r3node-rust` client over gRPC.
2. **`f1r3node-async-rust` client.** An async Rust library (and thin CLI) that human and agentic clients use to deploy to the shard. It signs the deploy, publishes it to a node's queue, and gives the caller an async handle for following the deploy's fate.

Three constraints shape the design.

- *No shared broker.* The queue is node-local. There is no central queue service whose failure takes down the network; a broker outage affects one node's intake only.
- *The deploy TTL stays, as a shard-wide parameter.* Deployments already expire after a block window. That window remains the mechanism that stops a backlog from becoming stale work; its length is set at shard launch, not compiled in.
- *Works on `dev` and on the cost-accounting branch.* The design must sit on current `dev` (run-to-completion with merge logic) and on `feature/rev-stacks-ote` (cost-accounted order-then-execute). It therefore lives at the deploy-ingress seam, above everything the two branches disagree about: execution, charging and finality.

Out of scope: changing deploy semantics, the consensus protocol, or how the proposer selects deploys. The queue changes *when* and *how fast* deploys reach the node, not what the node does with them.

2 Where this sits in the architecture

Figure 1 is the "F1R3FLY.io Deployment Architecture" diagram made concrete. The DeployQueue beside each F1R3Node is a RabbitMQ broker that only that node consumes from; the RustAsync-Client box is the `f1r3node-async-rust` library in the client's process. Everything to the right of the queue—consensus, RSpace, oracle-mode data sources such as SWI-Prolog and FileIO—is unchanged, so the diagram's invariant holds: all client traffic still enters through a node, consensus-mode data keeps one view, and oracle-mode data stays tied to the node that touched it.

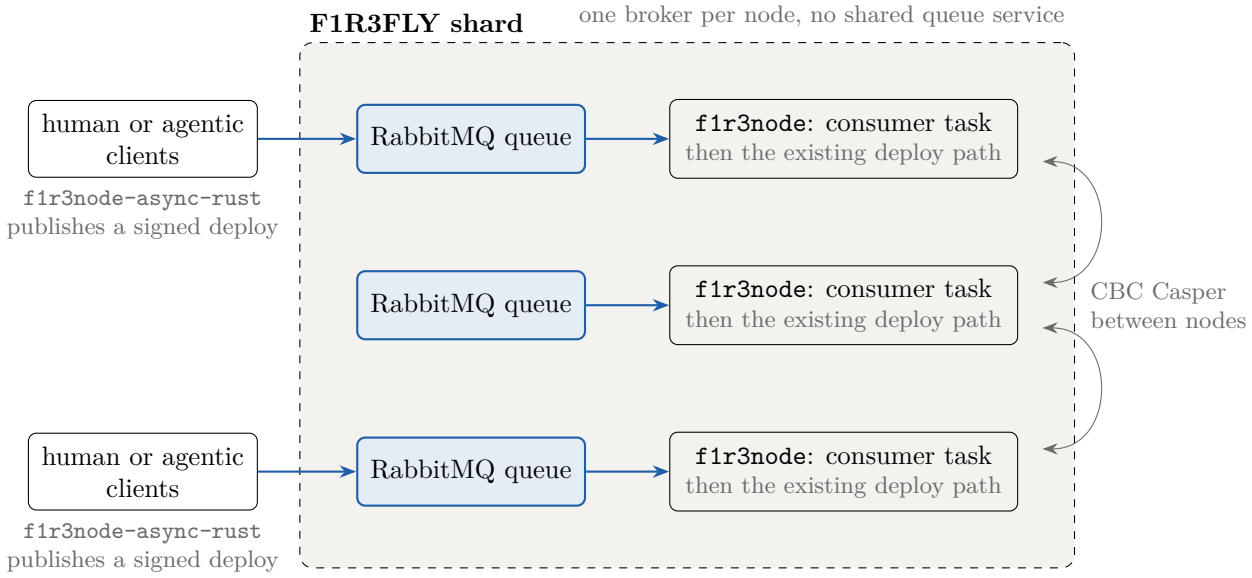


Figure 1: Each node owns its queue; clients publish to a node, consensus runs between nodes. A broker outage affects only its own node's intake.

3 Today's deploy ingress on `dev`

Every deploy, however it arrives, converges on one function: `BlockAPI::deploy` in `casper/src/rust/api/block_api.rs`. That is the seam the queue consumer will call. There is no buffering, rate limiting or queueing on the ingress path today, and nothing in the code or `docs/` mentions RabbitMQ or AMQP.

Entry points.

Entry	Where	What it does
gRPC `doDeploy`	`node/src/rust/api/deploy_grpc_service_v1.rs`, `do_deploy`	`DeployData::from_proto` (verifies the signature), then `BlockAPI::deploy`
HTTP `POST /api/deploy`, `/api/v1/deploy`	`node/src/rust/web/shared_handlers.rs` → `WebApiImpl::deploy` → `to_signed_deploy`	Same checks; JSON body `DeployRequest {data, deployer, signature, sigAlgorithm}`
CLI `node deploy`	`casper/src/rust/util/comm/deploy_runtime.rs`, `grpc_deploy_service.rs`	Builds and signs a `DeployData`, calls gRPC `doDeploy`; the only existing Rust client code

The wire type. `DeployDataProto` (`models/src/main/protobuf/CasperMessage.proto`) carries `deployer`, `term`, `timestamp`, `sig`, `sigAlgorithm`, `phloPrice`, `phloLimit`, `validAfterBlockNumber`, `shardId`, `language`, `expirationTimestamp`, `parameters`. The signature (`crypto/src/rust/signatures/signed.rs`, `Signed<DeployData>`) covers the proto encoding of the data fields only; `deployer`, `sig` and `sigAlgorithm` are outside the preimage, and `language` is dropped on decode. The deploy's identity everywhere is its signature (`DeployId = sig`).

Checks at ingress, in order (`BlockAPI::deploy`, then `admit_deploy` → `reserve_deploy` in `casper/src/rust/engine/multi_parent_casper/block_admission.rs`):

1. node not read-only (a node without a validator key refuses all deploys);
2. `shardId` equals the node's `casper.shard_name`;
3. deployer is not a system key;
4. wall-clock `expirationTimestamp`, if set, not passed;
5. Casper engine available (`engine_cell.get().await`; blocks until the approved block is in);
6. `phloPrice` $\geq$ `min-phlo-price` (the shard floor, read from chain);
7. local validator is bonded;
8. **block window:** `deploy_window::is_open(validAfter, latest+1, deploy_lifespan)`, i.e. $validAfter > latest + 1 - lifespan$;
9. not already known: pool (`deploy_storage.contains_sig`), DAG (`deploy_canonical_appearance`), rejected-deploy buffer;
10. term parses (`interpreter_util::mk_term`; failure → `DeployError::ParsingError`);
11. compare-and-swap insert into the pool with the signature re-verified (`prepare_checked`, `add_prepared_if_unchanged`); `AlreadyPresent` → `DuplicateDeploy`;
12. optional heartbeat wake, and a deploy-triggered propose when autopropose is on (up to 4 attempts, 250 ms apart).

Steps 1–4 and 6–10 are terminal verdicts: the same bytes will fail again. Step 5 (engine not yet available) and, operationally, step 7 (not bonded) are transient. That split is what the consumer's acknowledgement policy is built on (§5).

Errors seen by a client. `DeployError` (`casper/src/rust/casper.rs`): `ParsingError`, `MissingUser`, `UnknownSignatureAlgorithm`, `SignatureVerificationFailed`, `DuplicateDeploy`; plus `DeployValidationError{message}` for steps 1–8. HTTP maps them in `shared_handlers::classify_error` (409 `duplicate_deploy`, 400 `illegal_argument`); gRPC wraps them in `ServiceError`.

The pool and the proposer. Accepted deploys sit in `KeyValueDeployStorage` (`block-storage/src/rust/deploy/`, LMDB, keyed by sig), beside a `rejected_deploy_buffer` of merge-rejected deploys awaiting re-proposal. The pool is node-local; deploys never gossip. `prepare_user_deploys` (`casper/src/rust/blocks/proposer/block_creator.rs`) reads the whole pool, drops future deploys ($validAfter \geq block_number$) and expired ones (block-window or wall-clock; expired deploys are deleted from the pool at this point), filters out deploys already in scope, orders by ($validAfter$, $timestamp$, sig), and takes up to `max-user-deploys-per-block` (128) within a 2 MiB byte budget and the `deploy-play-budget` wall-clock budget, with tighter caps under finality-lag back-pressure. Deploys leave the pool when the finalizer settles them or when they expire.

Ingress limits that exist today. gRPC max message 16 MiB, HTTP body 16 MiB, tonic `concurrency_limit_per_connection(1024)`, and a proposer queue cap of 1 024 pending proposals. Nothing bounds the pool itself on `dev`.

Node start-up. `NodeRuntime::node_program` (`node/src/rust/runtime/node_runtime.rs`) owns a `JoinSet` of named critical tasks (`spawn_named_task`): transport, discovery, Casper loop, block processor, proposer, heartbeat proposer, gRPC/HTTP servers. Dependencies are wired in `node/src/rust/runtime/setup.rs::setup_node_program`, where `engine_cell`, `trigger_propose_f` and `shard_name`—the three things `BlockAPI::deploy` needs—are built. A queue consumer is one more named task wired there.

4 What changes on the cost-accounting branch

The ingress seam is the same on `feature/rev-stacks-ote` (tip `4c9853b`, which last merged `dev` at `3b5daabb4` on 2026-11-06): gRPC and HTTP both call `BlockAPI::deploy`, which runs the same twelve checks. Funding is *not* checked at ingress on either branch; the gate, precharge and static COMM bound run at proposal and execution only. So a consumer that calls `BlockAPI::deploy` behaves correctly on both branches without knowing which one it is on. Five differences matter.

Difference	`dev`	`feature/rev-stacks-ote`	Consequence for the queue
Pool cap	none	`pool_admission`: refuses when pending $\geq$ `max-user-deploys-per-block` $\times$ `deploy-pool-blocks-of-backlog` ($128 \times 64 = 8\,192$); HTTP 503 `deploy_pool_full`, `Retry-After: 5`; `DeployError::PoolFull` (P0-09, P0-10)	`PoolFull` is the back-pressure signal the consumer pauses on. `dev` has no such signal, so the consumer needs its own in-flight bound there, or P0-09 is ported to `dev` (§11)

Difference	`dev`	`feature/rev-stacks-ote`	Consequence for the queue
Wire type	fields 1–14	adds `cosignatures = 15`, `requiredCosigners = 16` (FX-3); the deployer signs `requiredCosigners`, cosigners sign a domain-separated preimage	The queue must carry the **raw proto bytes**. Decoding with `dev`'s type and re-encoding would drop fields 15–16 and break every cosigned envelope. Ingress also runs `with_complete_cosignatures`, so incomplete envelopes are terminal rejections
Repeat-deploy check	at block acceptance (`Validate::repeat_deploy`)	moved to execution: the gate's `AlreadyIncluded` and `Discarded` refusals, derived from canonical records and persisted per signature (FX-17, Decision 109)	A signature refused as `AlreadyIncluded`/`Discarded` is dead network-wide for its window; a retry needs a re-signed deploy (new timestamp ⇒ new sig). The client must never auto-retry the same bytes after a terminal verdict
Proposal trigger	`trigger_propose(casper, true)` with retries	`ProposeCoalescer` / `request_deploy_propose`; concurrent arrivals collapse into one extra proposal (P0-01)	The consumer passes the same `trigger_propose_f` each branch's HTTP handler passes; nothing queue-specific
Offer fate	`GET /api/deploy-finalization-receipt/{sig}`	plus `GET /deploy/offered/{id}/{sig}` → unknown / pending / rejected (reason, root, block) / ordered / notRetained / executed (P1-13). An offer refused by the gate screen at proposal is *removed from the pool*; the same signed deploy may be resubmitted	The client's status model gains one terminal-with-retry state on v6: *rejected at gate* (resubmit same bytes) versus `AlreadyIncluded`/`Discarded` (resign)

Selection on v6 (`v6/propose.rs::select_envelopes`) orders by (*timestamp*, *sig*), not by `phloPrice`, requires `phloLimit` > 0, *validAfter* $<$ *next* and *validAfter* $>$ *next* $-$ *lifespan*, packs up to `max_envelopes` (256) within the `demand_cap`, and then the gate screen drops unfunded offers. Envelopes are withheld entirely while the execution-lag gate (32), the unattested bound (128) or floor pacing (128) is tripped; offers simply wait in the pool. None of this changes what the queue does, but a buffered deploy on v6 can wait in the pool as well as in the queue, and both waits count against the same block window.

Elaboration is not an ingress check. The ingress parse (`mk_term`) does not run the elaboration pass that puts a term into the normative seal forms (FX-21); that happens in the interpreter at execution. A term that parses but fails elaboration becomes an unreadable envelope in a (c) record and is marked `Discarded` network-wide. An optional early elaboration check in the consumer (calling `source_to_adt_elaborated`) would turn that into an immediate terminal rejection; it is listed as a decision in §11.

Shard-wide parameters on v6 come in two forms, both fixed at genesis and read back from the approved block's post-state: (a) consensus parameters in `PoS.rhox`/`PoSV6.rhox` served by `getConsensusParameters` (`max-parent-depth`, `deploy-lifespan`, `min-phlo-price`), adopted unconditionally in `hash_set_casper`; and (b) the `ShardResourcePolicy` (`casper/src/rust/v6/resource_policy.rs`: `block_records` 64, `block_record_bytes` 8 MiB, `max_envelopes` 256, `demand_cap`, byte-to-phlo coefficients) embedded in `TokenMetadata.rhox` and read

by v6/genesis_policy.rs. Two gaps the survey found, outside this note's scope: approve_block_protocol.rs always passes ShardResourcePolicy::default() (operators cannot set it yet), and the Oct 26 search caps and time budget T are not in the code.

5 Component 1: the node + RabbitMQ pairing

Each node runs beside one RabbitMQ broker that nobody else consumes from. The node hosts a DeployQueueConsumer task that registers an AMQP consumer on the node's deploy queue, receives each delivery as the broker pushes it, and calls BlockAPI::deploy with the deploy exactly as the gRPC handler would. The queue is a buffer in front of the existing ingress, not a new ingress: every check in §3 still runs, in the same order, on the same code.

Topology on the broker (declared idempotently by the node at start-up, so a fresh broker needs no provisioning).

Object	Name (default)	Settings	Purpose
Exchange	f1r3node.deploy	direct, durable	Clients publish here with routing key = the node's queue name
Queue	f1r3node.deploy.<shard>.<pk-prefix>	durable, classic (single broker); x-max-length = deploy-queue.max-length, x-overflow = reject-publish; x-message-ttl derived (§6); dead-letter exchange f1r3node.deploy.dlx	The buffer. reject-publish makes an over-full queue nack the publisher instead of growing without bound; that nack is the burst signal clients see
Exchange	f1r3node.deploy.status	topic, durable	Node publishes verdicts and lifecycle events; routing key <deployer-pk-hex>.<sig-hex>, plus shard.tip for block-height heartbeats
Exchange + queue	f1r3node.deploy.dlx, f1r3node.deploy.expired	durable, bounded	Messages that hit the queue-side TTL land here, for operators and the soak harness

Message contract. Body = the raw DeployDataProto bytes, not a re-encoding. That keeps the node's proto the only decoder, so dev decodes fields 1–14 and feature/rev-stacks-ote also reads 15–16, and a client built against the v6 proto can publish to either. Properties: content_type = application/x-protobuf, message_id = <sig-hex>, app_id = f1r3node-async-rust/<version>, timestamp, delivery_mode = persistent; headers x-f1r3-schema = 1, x-f1r3-shard, x-f1r3-deployer. The headers are advisory—the consumer trusts only what it decodes and verifies—and exist so operators can inspect a queue without decoding bodies.

The consumer task. Built in setup.rs with engine_cell, trigger_propose_f, shard_name and the new deploy-queue config section; registered in node_program with spawn_named_task("Deploy Queue Consumer", ...). Library: lapin 4.x (async AMQP 0-9-1, tokio by default, publisher confirms, automatic connection recovery with replay of declarations and consumers; MIT). Behaviour:

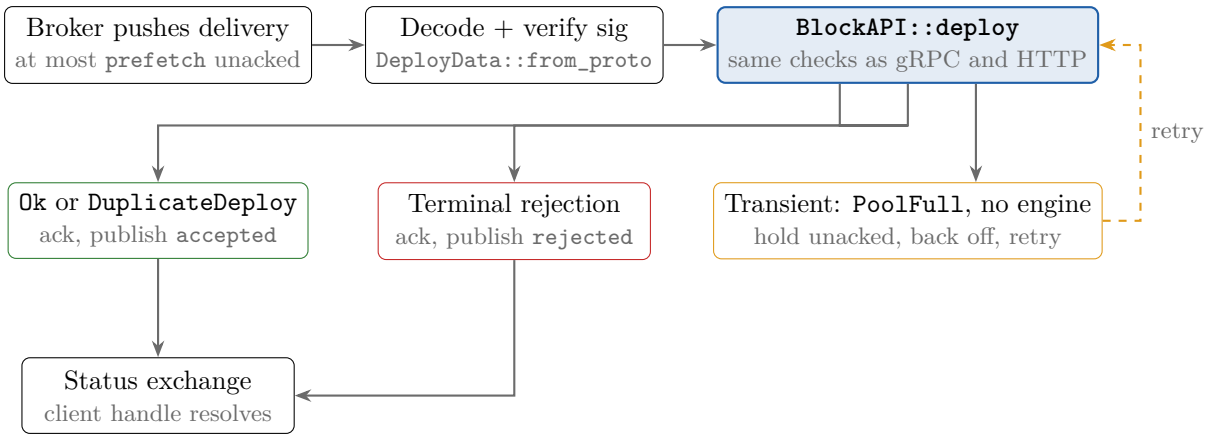
1. connect with `enable_auto_recover()`; declare the topology above; `basic_qos(prefetch)` so the broker pushes at most `prefetch` unacknowledged deliveries; `basic_consume` with manual acks;
2. for each delivery, decode `DeployDataProto`, then `DeployData::from_proto` (signature verified, cosignatures checked on v6), then `BlockAPI::deploy(engine_cell, signed, trigger_propose_f, read_only, shard_name)`—the same call `deploy_grpc_service_v1.rs` makes on `dev` and `shared_handlers.rs` makes on the branch. Up to `prefetch` deliveries are processed concurrently; `BlockAPI::deploy` is already safe under concurrent callers (the compare-and-swap insert in `reserve_deploy`);
3. classify the outcome and act as Table 1 says. Every outcome, including success, is published to the status exchange.

Outcome of `BlockAPI::deploy`	Class	Queue action	Status event
`Ok(deploy_id)`	accepted	`ack`	`accepted{sig, node, height}`
`DuplicateDeploy`	accepted earlier (redelivery after a crash, or a client retry)	`ack`	`accepted{..., duplicate}`
bad bytes, `SignatureVerificationFailed`, `UnknownSignatureAlgorithm`, incomplete cosignatures, shard mismatch, system key, `ParsingError`, wall-clock or block-window expired, `phloPrice` below floor, `MissingUser`	terminal	`ack` (consumed; never redelivered)	`rejected{sig, reason, terminal}`
engine not yet available, `PoolFull` (branch only), storage error	transient	leave **unacked**, back off (exponential, 250 ms → `max-backoff`), retry the same delivery; the prefetch window stops the broker pushing more, so the queue absorbs the burst while the node catches up	`deferred{sig, reason}` at most once a minute per sig
local validator not bonded, node read-only	transient with alarm	as transient, plus `warn!` and the `deploy_queue_stalled` metric; an operator problem, not a client one	`deferred{...}`

Table 1: Acknowledgement policy. Only the outcome of `BlockAPI::deploy` decides the queue action; the consumer never invents a verdict of its own.

A delivery is acked only after `BlockAPI::deploy` has returned, so a node crash between the call and the ack causes one redelivery, which hits `DuplicateDeploy` and is acked: at-least-once delivery plus signature-keyed idempotence gives exactly-once admission. Figure 2 shows the flow; Table 2 lists the failure modes.

Ordering. The queue is FIFO per publisher and the consumer processes up to `prefetch` deliveries concurrently, so arrival order into the pool is approximate. That is fine: on `dev` the



Held-unacked deliveries fill the prefetch window and the broker stops pushing: the queue absorbs the burst.

Figure 2: A delivery is acked only after the node has given a verdict.

proposer orders by (*validAfter*, *timestamp*, *sig*) and on v6 by (*timestamp*, *sig*), neither by arrival, and there are no per-deployer nonces. A client that needs two deploys in a fixed order waits for the first to be `accepted` (or further) before publishing the second, as it must today.

Back-pressure, end to end. Three bounds, outermost first: `x-max-length` with `reject-publish` (the client's publisher confirm comes back as a nack and the client surfaces `QueueFull`); the consumer's `prefetch` (how far ahead of the node the broker may push); and on the branch `PoolFull` from `pool_admission` (how full the node lets its pool get). On `dev` the third bound does not exist, so the consumer adds a soft one: it counts `list_pending_deploys()` (`dev` has no `pending_count`; the branch's `KeyValueDeployStorage::pending_count` arrives with DQ-2) against `deploy-queue.max-pool-backlog` at most once a second, and treats overflow as transient. Porting P0-09's `pool_admission` to `dev` would make the two branches identical here and is recommended.

6 Deploy TTL as a shard-wide parameter

The TTL is already shard-wide on both branches, so no new consensus parameter is needed. `deploy-lifespan` (default 50 blocks; `casper/src/rust/casper_conf.rs`, `defaults.conf`) only seeds genesis: it is written into the PoS contract (`$$deployLifespan$$` in `PoS.rhox` and `PoSV6.rhox`, via `genesis/contracts/standard_deploys.rs`), served by `PoS("getConsensusParameters")`, and at start-up `hash_set_casper` (`casper/src/rust/casper.rs`) reads it from the approved block's post-state through `token_metadata_check::read_on_chain_consensus_parameters` and overwrites the local value unconditionally. A node whose chain state lacks it fails to start. Every node in a shard therefore enforces the same window, and an operator sets it once, at shard launch, through `casper.deploy-lifespan` on the genesis node. The glossary (`docs/casper/GLOSSARY.md#deploy-lifespan`) deliberately avoids the word TTL because the window is counted in blocks, not time; this note uses "block window" for the consensus rule and "TTL" only for the queue-side wall-clock bound.

What the window means for a buffered deploy. The client stamps $validAfterBlockNumber = h$ when it publishes. The deploy is proposable while $h < next$ and $h > next - lifespan$, and ingress refuses it once $latest + 1 - lifespan \geq h$. Time spent in the queue, time spent in the pool, and (on v6) time withheld by the lag gate all count against the

Failure	Effect	Handling
Broker down or unreachable	This node stops taking queued deploys; gRPC/HTTP ingress unaffected; other nodes unaffected (no shared broker)	lapin auto-recovery with backoff; `deploy_queue_connected` gauge = 0; the task is supportive, not critical: it does not take the node down (§11)
Node down, broker up	Queue keeps accepting publishes up to `x-max-length`; deliveries wait	On restart the consumer resumes; deploys whose window closed meanwhile are terminal-rejected at ingress and reported
Both down (same host)	Clients get connection errors	Client policy (§7)
Poison message (undecodable)	Would loop if nacked with requeue	Classified terminal: acked, `rejected` event, counted in `deploy_queue_rejected_total` with `reason="decode"`
Queue-side TTL hit	Dead-lettered to `f1r3node.deploy.expired`	`expired{sig, stage="queue"}` published by the DLQ watcher; the deploy would have failed the block window anyway
Replay by a hostile publisher	Signature valid, bytes identical	`DuplicateDeploy` or block-window expiry; nothing to gain

Table 2: Failure modes and their handling.

same window. So a burst the shard cannot drain within *lifespan* blocks sheds its oldest deploys by design, each with a `rejected` or `expired` event rather than silently. That is the intended protection against overwhelming the shard: the buffer makes bursts survivable, the window keeps them bounded.

Where the window is enforced (same on both branches unless noted): ingress (`BlockAPI::deploy`); the proposer (`prepare_user_deploys` on `dev`, `select_envelopes` on v6, both deleting or skipping expired deploys); block validation (`Validate::transaction_expiration`, `future_transaction`, `time_based_expiration`); the merger (`dag_merger::split_window_closed_chains`); and on v6 the gate's prior-status window (`v6/core.rs::prior`). The queue adds no enforcement point; it only needs to stop holding messages that can no longer pass.

Queue-side TTL, derived not configured. RabbitMQ can expire messages by wall-clock age (`x-message-ttl`). The node derives it at start-up as

$$\mathrm{ttl}_{\mathrm{ms}} \;=\; \mathit{deploy_lifespan} \times \mathit{heartbeat.check_interval}_{\mathrm{ms}} \times \mathit{deploy_queue.ttl_factor},$$

with `ttl-factor` defaulting to 1.0 and 0 meaning "no queue-side TTL". `heartbeat.check-interval` (default 5 s) is the shard's block cadence under load; it already sizes `deploy-play-budget` the same way. The TTL is a housekeeping bound so a dead node's queue does not fill with deploys that are certainly expired; correctness never depends on it, because ingress re-checks the window on every delivery. Expired messages dead-letter to `f1r3node.deploy.expired`, where the DLQ watcher reports them.

Exposing the window to clients. The consumer publishes a `shard.tip` event on the status exchange on every new block it learns of: `{shard, block_number, deploy_lifespan, min_phlo_price, max_user_deploys_per_block, node}`. A subscribed client needs no HTTP

call to choose `validAfterBlockNumber` or to know how long its deploy can live. The same values are available from the node's existing HTTP endpoints, so a client without a status subscription still works.

Changing the window later. Because the value lives in the PoS contract, changing it on a running shard is a governance action (an updatable hook or a new genesis), not a config edit, on both branches—consistent with the principle that shard-wide parameters are set at launch and changed through multisig-authorised hooks.

7 Component 2: the `f1r3node-async-rust` client

A Rust library crate (`f1r3node-async-rust`, with a thin `f1r3deploy` binary over it) that signs a deploy, publishes it to one node's queue, and returns a handle whose futures resolve as the deploy's status events arrive. It is fully async (tokio), never blocks on the node, and never talks to the node's gRPC or HTTP ingress for the deploy itself. It depends on the `models` and `crypto` crates for `DeployData`, `Signed` and the signature algorithms, so signing is byte-identical to `node deploy` and to `deploy_runtime.rs` today.

Public API (sketch).

```
let signer = Secp256k1Signer::from_hex(&private_key_hex)?;
let client = NodeClient::connect(NodeEndpoint {
    amqp: "amqps://deployer@validator-1.example:5671/%2f".parse()?,
    shard_id: "asi-testnet".into(),
    queue: QueueName::default_for("asi-testnet", &validator_pk),
    status: StatusSubscription::Deployer,    // <pk>.# on the status exchange
}).await?;

let handle: DeployHandle = client
    .deploy(DeployRequest {
        term,
        phlo_limit: 100_000,
        phlo_price: PhloPrice::AtLeastFloor,  // or Exact(n)
        valid_after: ValidAfter::LatestTip,   // from the last shard.tip; or
    Explicit(h)
        expiration_timestamp: None,
        cosigners: vec![],                    // v6: required cosigners
    }, &signer)
    .await?;                                  // resolves on the broker's publish
    confirm

handle.id();                                  // DeployId = sig bytes, hex
handle.accepted().await?;                     // Accepted | Rejected(reason) |
    Deferred(reason)
let mut events = handle.events();             // Stream<DeployEvent>
handle.finalized().await?;                    // Ok(receipt) | Err(Expired |
    Rejected | Discarded)
```

Publish semantics. `deploy()` opens a confirm-mode channel and publishes with `mandatory = true` and `delivery_mode = persistent`. It resolves `Ok(handle)` on a broker ack, `Err(QueueFull)` on a nack caused by `x-max-length` with `reject-publish`, `Err(Unroutable)` if the node has not declared its queue yet (the exchange returns the message), and `Err(Broker(_))` on connection failure. None of these errors means the deploy was seen by the node, so all are safe to retry with the same bytes. After a broker ack the deploy is durably queued; a client

crash from then on loses nothing, and the handle can be reconstructed from the signature (`client.attach(deploy_id)`).

Status tracking. The handle is driven by the status exchange. On connect the client declares an exclusive, auto-delete queue bound to `f1r3node.deploy.status` with routing key `<deployer-pk-hex>.#` (one subscription covers all of a deployer's deploys) and to `shard.tip`.

Event	Source	Terminal?	`dev`	v6
`accepted`	consumer, after `BlockAPI::deploy`	no	yes	yes
`rejected{reason, terminal}`	consumer	yes	yes	yes
`deferred{reason}`	consumer	no	yes	yes
`proposed{block}`	node lifecycle hook	no	phase 2	phase 2
`gate_rejected{reason}` (offer removed from pool; same bytes may be resubmitted)	node lifecycle hook	yes, retryable	n/a	phase 2
`finalized{receipt}` / `expired` / `discarded`	node lifecycle hook (`finality/deploy_lifecycle.rs` verdicts)	yes	phase 2	phase 2
`shard.tip`	consumer, per new block	n/a	yes	yes

Phase 1 ships `accepted`, `rejected`, `deferred` and `shard.tip`, which the consumer can emit without touching Casper internals. For everything past admission the phase-1 handle polls the node's existing HTTP endpoints at a slow cadence (`/api/deploy-finalization-status/{sig}` on both branches; `/deploy/offered/{id}/receipt` on v6) until phase 2 adds lifecycle events from the finalizer. The handle's API does not change between phases.

Retry policy, built in and conservative. The client retries only publish-side failures with the same bytes. After a terminal `rejected`, `discarded` or `expired` it never republishes the same signature: on v6 an `AlreadyIncluded`/`Discarded` signature is dead for its window (FX-17), and on both branches a block-window rejection means a new `validAfterBlockNumber`, hence a new signature. `handle.resign_and_retry(&signer)` is the explicit, opt-in path that builds a fresh deploy (new timestamp and `validAfter`) and publishes it. `gate_rejected` on v6 is the one retryable-with-same-bytes terminal state; the client exposes `handle.resubmit()` for it.

Choosing a node. The queue is node-local and the default client talks to exactly one node. `ShardClient` is a thin optional layer over several `NodeClient`s with one policy: sticky to a preferred node, failing over to the next on `Broker`/`Unroutable` errors—never on `QueueFull`, which is the shard telling the client to slow down. It does not spread one client's deploys across nodes; a deploy's status subscription lives on the node that took it.

Cosigned deploys (v6). `client.prepare(request, &signer)` returns the envelope awaiting cosigners; `cosign(envelope, &cosigner)` adds a `DeployCosignature` using the FX-3 preimage; `deploy_envelope(envelope)` publishes. The CLI mirrors the branch's `node cosign prepare | sign | submit`.

Binary. `f1r3deploy -node amqps://... -shard asi-testnet -key-file k.pem -phlo-limit 100000 file.rho [-wait accepted|finalized]` prints the deploy id and, with `-wait`, the terminal event as JSON. A `f1r3deploy burst -clients N -rate R -duration D` subcommand drives the soak test in §10.

Where it lives. A new repository `F1R3FLY-io/f1r3node-async-rust`, Apache 2.0 like the node, depending on `f1r3node-rust` crates by git revision. Alternative: a `client/` workspace crate inside `f1r3node-rust`, which keeps versions in lock-step but ties client releases to node releases (§11).

8 Branch compatibility

One implementation serves both branches because everything branch-specific sits behind `BlockAPI::deploy` and the `DeployDataProto` decoder, and the consumer touches neither.

Concern	Shared	Per branch
Consumer task, topology, ack policy, status publisher, config section	yes	—
Decoding the body	`prost` decode of `DeployDataProto` from the branch's own `models` crate	`dev` ignores unknown fields 15–16; the branch reads them
Signature and cosignature verification	`DeployData::from_proto`	the branch's `from_proto` also runs `with_complete_cosignatures`
The admission call	`BlockAPI::deploy(engine_cell, signed, trigger_propose_f, read_only, shard_name)`	argument list identical on both; the branch's `trigger_propose_f` is the coalescing one
Outcome classification	one `classify(&err) -> Class` function	`PoolFull` exists only on the branch; on `dev` the arm is unreachable and the soft pool bound stands in
Status events	same schema	`gate_rejected`, `discarded`, receipts only on v6 (phase 2)

Mechanically: the patch is written once against `dev` and cherry-picked onto `feature/rev-stacks-ote` (or its successor integration branch). The branch's `block_api.rs` has moved lines but the same signature, so the conflict surface is `setup.rs` wiring and `defaults.conf`. The client crate builds against either branch's `models`/`crypto` at a pinned revision; because the branch's proto is a strict superset, a client built against the branch can publish to a `dev` shard (fields 15–16 empty) and a client built against `dev` can publish uncosigned deploys to a v6 shard. The design deliberately does *not* reach into v6 internals (gate, policy, seals): a buffered v6 deploy still meets the gate at proposal, exactly as an unbuffered one does, so nothing here needs to change when the cost-accounting branch lands in `dev`.

9 Operations

Configuration — a new node-local section in `defaults.conf` / `NodeConf` (not a consensus parameter; per-node values may differ, like `deploy-play-budget`):

```
deploy-queue {
  enabled = false                 # off by default; gRPC/HTTP ingress unchanged
  amqp-url = "amqp://127.0.0.1:5672/%2f"
  queue-name = ""                 # "" = f1r3node.deploy.<shard>.<pk-prefix>
  prefetch = 64                   # unacked deliveries the broker may push
  max-length = 100000             # x-max-length; overflow = reject-publish
  ttl-factor = 1.0                # x-message-ttl = lifespan * check-interval *
    factor; 0 = none
  max-pool-backlog = 8192         # dev only: soft bound (branch uses
    pool_admission)
```

```
  max-backoff = 30 seconds        # retry ceiling for transient outcomes
  status-exchange = "f1r3node.deploy.status"
  tls { enabled = false, ca-file = "", cert-file = "", key-file = "" }
}
```

CLI overrides follow the `config_mapper.rs` pattern for each key. `enabled = true` with an unreachable broker logs an error and retries; it does not stop the node.

Running the broker. One RabbitMQ container per node: `docker/shard.yml` gains a `rabbitmq-<n>` service per validator, with the management plugin bound to localhost only; `docker/standalone.yml` gains one. The `ign1t10n` installer (local macOS shard) adds a RabbitMQ install step and the three config lines. Production guidance: broker and node on the same host or pod so "node-local" holds physically; a broker shared between validators is a configuration error the node detects at start-up (queue name already bound to another validator's consumer tag) and refuses.

Security. The deploy is signed end to end, so a compromised broker can delay, drop or replay but not forge; replay is idempotent. AMQP credentials: one publishing user per shard (or per client organisation) with `write` on `f1r3node.deploy` and `read` on `f1r3node.deploy.status` only; the node's user alone has `configure`. TLS (`amqps`) on any non-loopback listener, reusing the node's TLS material. Status events carry no term text, only signatures, reasons and block references.

Metrics (Prometheus, beside the node's existing ones): `deploy_queue_connected`, `deploy_queue_depth`, `deploy_queue_consumed_total{outcome}`, `deploy_queue_rejected_total{reason}`, `deploy_queue_deferred_seconds` (time a delivery waited unacked), `deploy_queue_admission_seconds` (`BlockAPI::deploy` latency), `deploy_queue_stalled`, `deploy_queue_dead_lettered_total`. A new `GET /api/deploy-queue` returns `{enabled, connected, depth, prefetch, max_length, ttl_ms, consumed_total}` so a client or operator can see the buffer without broker credentials.

10 Development plan

Five patches, each independently testable, in the lane style of the rev-stacks plan (`patch/<ID>` branches, `[agent]` commits, decisions logged under `docs/designs/deploy-queue/decisions/`). DQ-1 and DQ-3 can run in parallel once the message contract in §5 is frozen.

Patch	Scope	Branch(es)	Acceptance
DQ-1 node consumer	`deploy-queue` config + CLI mapping; `DeployQueueConsumer` (lapin), topology declaration, classify + ack policy, `shard.tip/accepted/rejected/deferred` publisher, metrics, `GET /api/deploy-queue`	`dev`, then cherry-pick to `feature/rev-stacks-ote`	With `enabled = false` behaviour is unchanged. With a broker up, a deploy published as raw proto is admitted and appears in `/api/pending-deploys`; redelivery after a kill between admission and ack yields `duplicate`, not a second pool entry; each terminal class produces ack + `rejected`; `PoolFull` (branch) and the soft bound (`dev`) hold the delivery unacked and the broker stops at `prefetch`
DQ-2 pool cap on `dev` (optional, recommended)	Port P0-09/P0-10 `pool_admission`, `PoolFull`, HTTP 503 `deploy_pool_full`	`dev`	The branch's P0-09 tests pass on `dev`; the soft bound becomes dead code and is removed
DQ-3 client crate	`f1r3node-async-rust`: `NodeClient`, `DeployRequest`, `DeployHandle`, status subscription, confirm-mode publishing, error taxonomy, `ShardClient` failover, `f1r3deploy` incl. `burst`	new repo, pinned to each branch	Signs byte-identically to `node deploy` (golden test against `deploy_runtime.rs`); `QueueFull`, `Unroutable`, `Broker` surfaced and retried correctly; `accepted()` resolves from the event stream; polling fallback resolves `finalized()` on both branches
DQ-4 packaging	`docker/shard.yml`, `docker/standalone.yml` broker services; `ign1t10n` step; operator page in `docs/node/`; glossary anchors `#deploy-queue`, `#status-exchange`	`dev`	`docker compose -f docker/shard.yml up` brings up three validators each with its own broker; `f1r3deploy` against any one of them succeeds
DQ-5 lifecycle events (phase 2)	Hook in `finality/deploy_lifecycle.rs` (both) and the v6 offer-status path to publish `proposed`, `gate_rejected`, `finalized`, `expired`, `discarded`	both	The handle resolves `finalized()` without polling; events match `/api/deploy-finalization-status` and `/deploy/offered/{id}/receipt` in a randomized comparison

Tests.

- Unit: the classification table (every `DeployError`/`CasperError` arm has a class); TTL derivation; queue-name derivation; proto round-trip with fields 15–16 present on `dev`'s decoder (must survive as raw bytes, never be re-encoded).

- Integration: RabbitMQ in `testcontainers`; the three-validator `docker/shard.yml`; crash between admit and ack; broker restart mid-burst (auto-recovery); node restart with a full queue.
- Soak, the point of the exercise: `f1r3deploy burst` from N clients against one node and against all three, at rates above and below what the shard drains, measuring admission latency, pool depth, time-to-finalized, and the fraction shed by the block window. Run on both branches through `scripts/casper-soak`; the v6 run is the one that matters for the ASI TestNet and should follow the Track B handoff candidate, not precede it. Pass criteria: no validator stall across the run; no deploy lost without a terminal event; shed fraction 0 at a sustainable rate and bounded by *lifespan* at overload.

Order. DQ-1 on `dev` first (the merge target), DQ-3 against it, DQ-4 to make the three-node test reproducible, then the cherry-pick of DQ-1 onto the cost-accounting branch and the v6 soak; DQ-2 and DQ-5 follow. Nothing merges to `dev` until the soak passes on both.

11 Open questions and decisions needed

Each has a recommended default so work can start; answers that differ change the patch, not the design.

1. **Broker outage and the node.** Recommended: the consumer is a supportive task; a broker failure degrades this node to gRPC/HTTP ingress and alarms, never stops the node. Alternative: treat it as critical so a buffered-only deployment fails loudly.
2. **Queue-only mode.** Should a shard be able to close gRPC/HTTP `doDeploy` so every deploy is buffered? Recommended: a `deploy-queue.exclusive = false` flag; when on, the direct endpoints return 503 `use_deploy_queue` with the queue name.
3. **Port the pool cap to `dev` (DQ-2).** Recommended yes: both branches then share the same back-pressure signal and the soft bound goes away.
4. **Status events in phase 1.** Recommended: ingress verdicts + `shard.tip` only; lifecycle events in DQ-5 after the Track B handoff, so the consumer does not touch the finalizer while that lane is landing.
5. **Client repository.** Recommended: a separate `F1R3FLY-io/f1r3node-async-rust` repo pinned to node revisions, since clients release on their own cadence and ASI's TestNet users will consume it independently of node builds. Alternative: a `client/` workspace crate.
6. **Early elaboration check on v6.** Running `source_to_adt_elaborated` in the consumer would reject malformed seal forms immediately instead of letting them become `Discarded` network-wide, at the cost of CPU per delivery and a dependency on the rholang crate's elaborator. Recommended: off by default, behind `deploy-queue.elaborate`, revisited after the v6 soak shows how often it would fire.
7. **Queue-side TTL.** Recommended: derived (`ttl-factor = 1.0`) rather than unset, so a dead node's queue cannot fill with certainly-expired deploys; `0` if operators prefer the ingress check to be the only expiry.
8. **Per-deployer fairness.** A single client can fill a node's queue and starve others until `x-max-length` pushes back on everyone; RabbitMQ has no per-publisher quota. Options: accept it (node-local queue, node operator chooses its clients); or per-deployer queues bound to the same exchange with the consumer round-robining across them. Recommended: accept it for phase 1, record the per-deployer variant as the follow-up if the soak shows starvation.
9. **Naming and home.** `f1r3node-async-rust` is the working name; `f1r3node-client` is the alternative. Also whether this note is checked into `docs/designs/deploy-queue/` on `dev` as the lane's design record.

Sources

- F1R3FLY-io/f1r3node-rust, dev at a79a6ce and feature/rev-stacks-ote at 4c9853b: casper/src/rust/api/block_api.rs, engine/multi_parent_casper/block_admission.rs, blocks/proposer/block_creator.rs, util/deploy_window.rs, casper.rs (hash_set_casper), v6/{propose,gate,core, resource_policy,genesis_policy}.rs, node/src/rust/runtime/{setup, node_runtime}.rs, node/src/main/resources/defaults.conf, casper/src/main/resources/PoS.rhox.
- docs/casper/GLOSSARY.md (#deploy-admission, #deploy-lifespan, #rejected-deploy-buffer, #deploy-lifecycle); docs/node/api-reference.md; docs/rholang/18-deployment-workflow.md; docs/data-flows/README.md.
- docs/rev-stacks/decisions/: FX-3, FX-8b, FX-17, FX-21, FX-22, P0-01, P0-09, P0-10, P1-13, P2-07, P2-10; docs/plans/v6-order-then-execute-tranche-1.md.
- "F1R3FLY.io Deployment Architecture" (PDF, Google Drive, 2026-11-07).
- lapin 4.10 documentation, https://docs.rs/lapin.